

RAMDEOBABA UNIVERSITY NAGPUR



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Student Name :- Advait Kawale

Section :- A4 (B3)

Roll No :- 42

Course Name :- Design and Analysis of Algorithms

Course Code :- 24CS01TH0201

Title :- Practical 4

Practical 4

Aim :- Implement maximum sum of subarray for the given scenario of resource allocation using the divide and conquer approach.

Ans)

Input)

```
#include <stdio.h>
int main()
{
    int n,constraint;
    printf("Enter size of array = ");
    scanf("%d",&n);
    int a[n];
    for (int i=0;i<n;i++)
    {
        printf("Enter element %d = ",i+1);
        scanf("%d",&a[i]);
    }
    printf("Enter constraint = ");
    scanf("%d",&constraint);
    if (n == 0)
    {
        printf("\nNo subarray");
        return 0;
    }
    if (constraint==0)
    {
        printf("\nNo subarray possible");
        return 0;
    }
    int l=0,r=-1,sum=0,min=0,max=-1,start=0;
    for(int i=0;i<n;i++)
    {
        sum+=a[i];
        while(sum>constraint&&start<=i)
        {
```

```

        sum-=a[start++];
    }
    if(sum<=constraint&&sum>max)
    {
        max=sum;
        l=start;
        r=i;
    }
}
if(r==-1 || max==0)
{
    printf("\nNo feasible subarray");
}
else
{
    printf("\nThe Maximum Sum Subarray = ");
    for(int i=l;i<=r;i++)
    {
        printf("%d ",a[i]);
    }
    printf("\nSum = %d",max);
}
}

```

```

1  #include <stdio.h>
2  int main()
3  {
4      int n,constraint;
5      printf("Enter size of array = ");
6      scanf("%d",&n);
7      int a[n];
8      for (int i=0;i<n;i++)
9      {
10         printf("Enter element %d = ",i+1);
11         scanf("%d",&a[i]);
12     }
13     printf("Enter constraint = ");
14     scanf("%d",&constraint);
15     if (n == 0)
16     {
17         printf("\nNo subarray");
18         return 0;
19     }
20     if (constraint==0)
21     {
22         printf("\nNo subarray possible");
23         return 0;
24     }
25     int l=0,r=-1,sum=0,min=0,max=-1,start=0;
26     for(int i=0;i<n;i++)
27     {
28         sum+=a[i];
29         while(sum>constraint&&start<=i)
30         {
31             sum-=a[start++];
32         }
33         if(sum<=constraint&&sum>max)
34         {
35             max=sum;
36             l=start;
37             r=i;
38         }
39     }
40     if(r== -1 || max==0)
41     {
42         printf("\nNo feasible subarray");
43     }
44     else
45     {
46         printf("\nThe Maximum Sum Subarray = ");
47         for(int i=l;i<=r;i++)
48         {
49             printf("%d ",a[i]);
50         }
51         printf("\nSum = %d",max);
52     }
53 }

```

Output)

1. Basic small array

- resources = [2, 1, 3, 4], constraint = 5
- o Best subarray: [2, 1] or [1, 3] → sum = 4
- o Checks simple working.

```

Enter size of array = 4
Enter element 1 = 2
Enter element 2 = 1
Enter element 3 = 3
Enter element 4 = 4
Enter constraint = 5

The Maximum Sum Subarray = 1 3
Sum = 4

```

2. Exact match to constraint

- resources = [2, 2, 2, 2], constraint = 4
 - o Best subarray: [2, 2] → sum = 4
 - o Tests exact utilization.

```
Enter size of array = 4
Enter element 1 = 2
Enter element 2 = 2
Enter element 3 = 2
Enter element 4 = 2
Enter constraint = 4

The Maximum Sum Subarray = 2 2
Sum = 4
```

3. Single element equals constraint

- resources = [1, 5, 2, 3], constraint = 5
 - o Best subarray: [5] → sum = 5
 - o Tests one-element solution.

```
Enter size of array = 4
Enter element 1 = 1
Enter element 2 = 5
Enter element 3 = 2
Enter element 4 = 3
Enter constraint = 5

The Maximum Sum Subarray = 5
Sum = 5
```

4. All elements smaller but no combination fits

- resources = [6, 7, 8], constraint = 5
 - o No feasible subarray.
 - o Tests "no solution" case.

```
Enter size of array = 3
Enter element 1 = 6
Enter element 2 = 7
Enter element 3 = 8
Enter constraint = 5

No feasible subarray
```

5. Multiple optimal subarrays

- resources = [1, 2, 3, 2, 1], constraint = 5
 - o Best subarrays: [2, 3] and [3, 2] → sum = 5
 - o Tests tie-breaking (should return either valid subarray).

```
Enter size of array = 5
Enter element 1 = 1
Enter element 2 = 2
Enter element 3 = 3
Enter element 4 = 2
Enter element 5 = 1
Enter constraint = 5

The Maximum Sum Subarray = 2 3
Sum = 5
```

6. Large window valid

- resources = [1, 1, 1, 1, 1], constraint = 4
 - o Best subarray: [1, 1, 1, 1] → sum = 4
 - o Ensures long window works.

```
Enter size of array = 5
Enter element 1 = 1
Enter element 2 = 1
Enter element 3 = 1
Enter element 4 = 1
Enter element 5 = 1
Enter constraint = 4

The Maximum Sum Subarray = 1 1 1 1
Sum = 4
```

7. Sliding window shrink needed

- resources = [4, 2, 3, 1], constraint = 5
 - o Start [4,2] = 6 (too big) → shrink to [2,3] = 5.
 - o Tests dynamic window adjustment.

```
Enter size of array = 4
Enter element 1 = 4
Enter element 2 = 2
Enter element 3 = 3
Enter element 4 = 1
Enter constraint = 5

The Maximum Sum Subarray = 2 3
Sum = 5
```

8. Empty array

- resources = [], constraint = 10
- o Output: no subarray.
- o Edge case: empty input.

```
Enter size of array = 0
Enter constraint = 10

No subarray
```

9. Constraint = 0

- resources = [1, 2, 3], constraint = 0
- o No subarray possible.
- o Edge case: zero constraint.

```
Enter size of array = 3
Enter element 1 = 1
Enter element 2 = 2
Enter element 3 = 3
Enter constraint = 0

No subarray possible
```